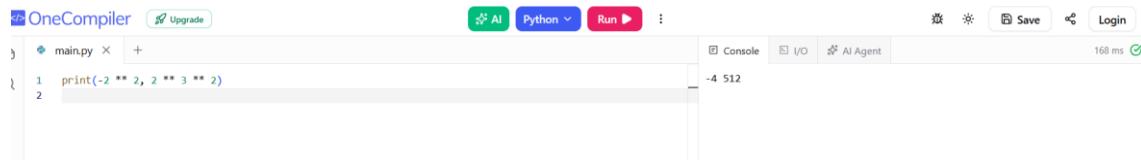


Aula007: Rômulo Pardim, Ricardo Henrique

Slide 4 - Quiz de abertura:

1 · Python



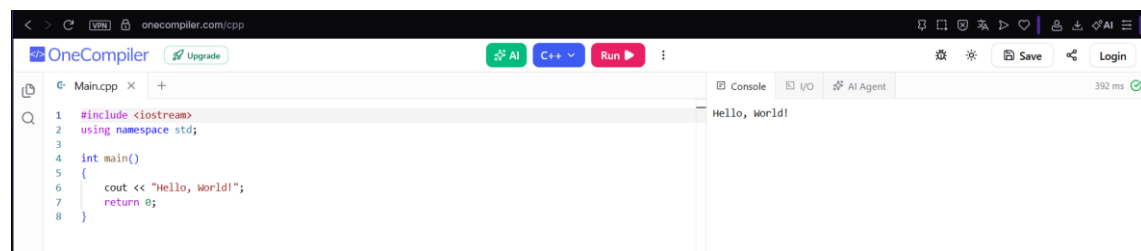
The screenshot shows the OneCompiler Python editor. The code in the editor is:

```
1 print(-2 ** 2, 2 ** 3 ** 2)
```

The output in the console is:

```
-4 512
```

4 · C



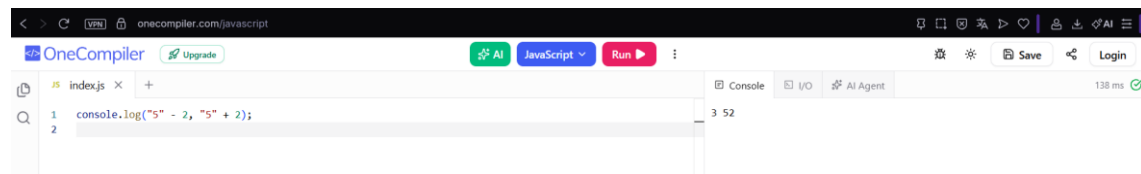
The screenshot shows the OneCompiler C++ editor. The code in the editor is:

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     cout << "Hello, World!";
7     return 0;
8 }
```

The output in the console is:

```
Hello, World!
```

3 · JavaScript



The screenshot shows the OneCompiler JavaScript editor. The code in the editor is:

```
1 console.log("5" - 2, "5" + 2);
```

The output in the console is:

```
3 52
```

Slide 7 - **Ordem dos operandos e efeitos colaterais**

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. · §7.2.2 · p. 307-309 · exemplos/02_efeitos_colaterais

Foi analisado como a ordem de execução pode alterar o resultado de uma expressão.

Slide 8 – Operadores sobrecarregados

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);      // 98
System.out.println("" + 'a' + 1); // a1
x.add(y).multiply(tres);          // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):
        # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k):
        # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)  # Vetor(4, 6)
print(v * 3)                  # Vetor(12, 18)
```

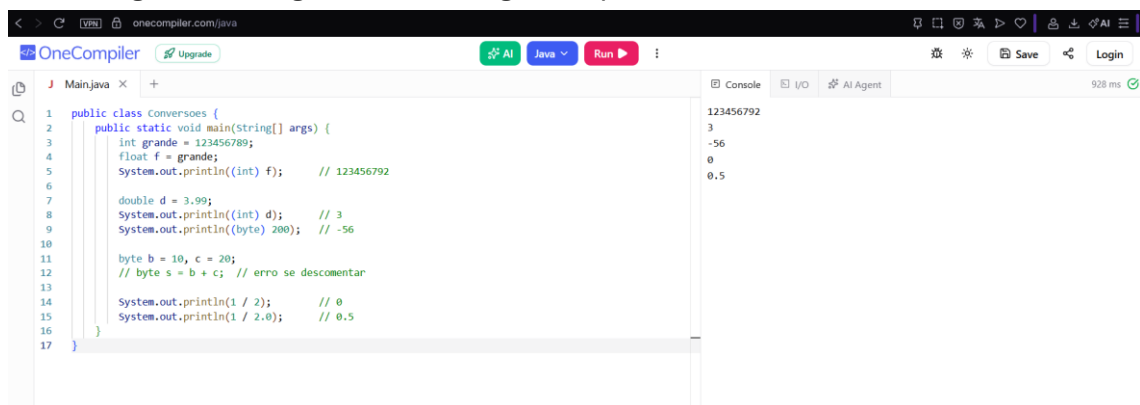
Sebesta, 11ª ed. · §7.3 · p. 309-310 e §9.11 · p. 404 · exemplos/03_sobrecarga

Foi estudado como um mesmo operador pode possuir diferentes funções dependendo dos tipos utilizados.

Slide 9 – Conversões de tipos: alargamento, estreitamento e coerção

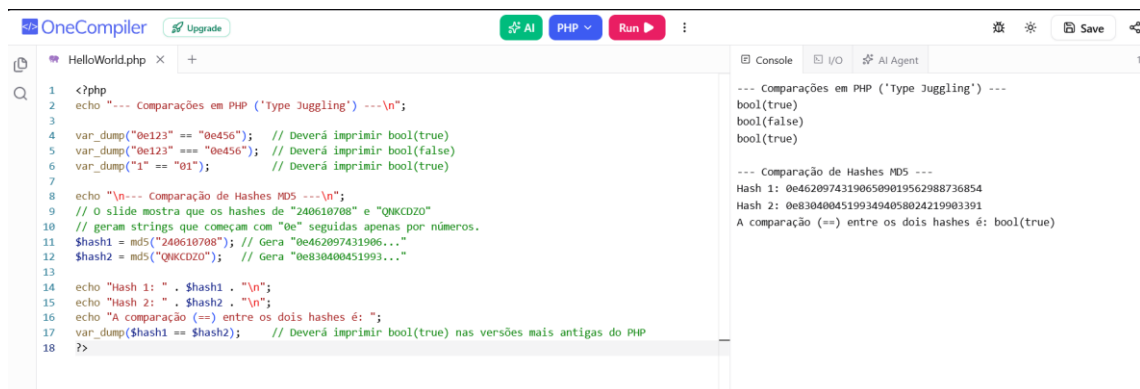
Conversões de tipos (Sebesta §7.4)

- **Alargamento:** vai para um tipo maior, em geral de forma implícita. Pode perder precisão: `int` → `float` transforma 123456789 em 123456792.
- **Estreitamento:** vai para um tipo menor e exige cast. Trunca ou estoura: (`int`) 3.99 dá 3, e (`byte`) 200 dá -56.
- **Coerção:** é a conversão automática. C, Java e Python fazem; Go, Rust, ML e F# não fazem e acusam erro de compilação.
- **Moral:** a coerção é cômoda, mas esconde erros. Em `d = b * a`, com a digitado no lugar de c, o código compila sem aviso.



```
1 public class Conversoes {
2     public static void main(String[] args) {
3         int grande = 123456789;
4         float f = grande;
5         System.out.println((int) f); // 123456792
6
7         double d = 3.99;
8         System.out.println((int) d); // 3
9         System.out.println((byte) 200); // -56
10
11        byte b = 10, c = 20;
12        // byte s = b + c; // erro se descomentar
13
14        System.out.println(1 / 2); // 0
15        System.out.println(1 / 2.0); // 0.5
16    }
17 }
```

Slide 10 - Coerção implícita: JavaScript e PHP



```
1 <?php
2 echo "---- Comparações em PHP ('Type Juggling') ----\n";
3
4 var_dump("0e123" == "0e456"); // Deverá imprimir bool(true)
5 var_dump("0e123" === "0e456"); // Deverá imprimir bool(false)
6 var_dump("1" == "01"); // Deverá imprimir bool(true)
7
8 echo "\n--- Comparação de Hashes MD5 ---\n";
9 // O slide mostra que os hashes de "240610708" e "QNKCDZO"
10 // geram strings que começam com "0e" seguidas apenas por números.
11 $hash1 = md5("240610708"); // Gera "0e462097431906..."
12 $hash2 = md5("QNKCDZO"); // Gera "0e830400451993..."
13
14 echo "Hash 1: " . $hash1 . "\n";
15 echo "Hash 2: " . $hash2 . "\n";
16 echo "A comparação (==) entre os dois hashes é: ";
17 var_dump($hash1 == $hash2); // Deverá imprimir bool(true) nas versões mais antigas do PHP
18 ?>
```

Para evitar resultados imprevisíveis causados pela conversão automática de tipos, utilize sempre os operadores de igualdade estrita (`===` e `!==`) em JavaScript e PHP, preterindo o operador de igualdade fraca (`==`).

Slide 11 - Expressões relacionais e booleanas

Linguagem

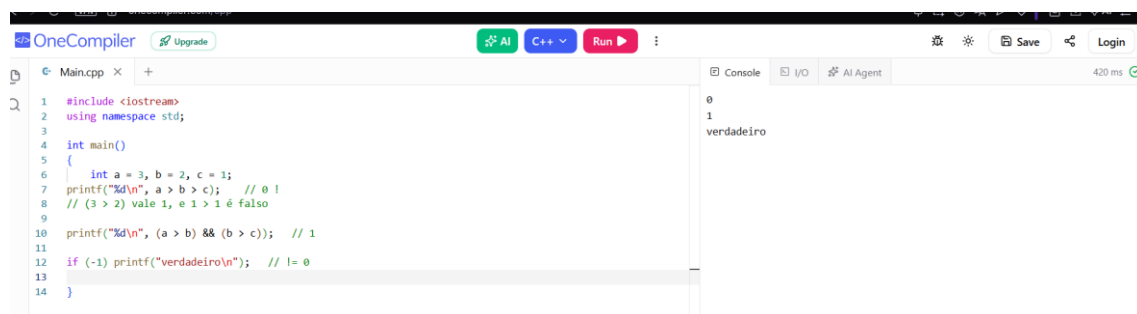
O que o if aceita

C, Python, JavaScript

qualquer valor (0, vazio e nulo contam como falso)

Java, C#, Go, Rust

somente booleano: if (n) com n inteiro nem compila



```
#include <iostream>
using namespace std;

int main()
{
    int a = 3, b = 2, c = 1;
    printf("%d\n", a > b > c); // 0 !
    // (3 > 2) vale 1, e 1 > 1 é falso
    printf("%d\n", (a > b) && (b > c)); // 1
    if (-1) printf("verdadeiro\n"); // != 0
}
```

Console output:

```
0
1
verdadeiro
```

Slide 12 - Avaliação em curto-circuito

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++; // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade // undefined, sem erro

const qtd = 0;
qtd || 10 // 10 (o 0 se perdeu!)
qtd ?? 10 // 0 (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em $(a > b) \ || \ (b++ \ / \ 3)$, o $b++$ só acontece quando $a \leq b$. Em Python, 0 or "padrão" devolve "padrão": and e or devolvem um dos **operandos**.

Foi estudado como operadores lógicos podem evitar a avaliação desnecessária de partes de uma expressão.

Slide 13 - Sentenças de atribuição

Sentenças de atribuição

C - a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... }          // compila! atribui e testa y
```

Python - atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: <code>if (x = y)</code> compila
Java, C#	sim, mas o <code>if</code> exige booleano
Python	não, exceto o operador morsa <code>:=</code>
Go	não: nem <code>x++</code> é expressão
Rust	vale <code>()</code> : <code>if x = 5</code> não compila

Linguagens funcionais (p. 321)
`val` (ML) e `let` (F#, Rust) dão **nome** a um valor, que não muda.

Sebesta, 11ª ed. - §7.7 - p. 317-321 - exemplos/06_atribuicao

- **A atribuição como expressão:** É possível identificar quais linguagens consideram a atribuição uma expressão que devolve um valor (C, C++, JavaScript, Java, C#) e as que não consideram (Python, Go, Rust).
- **Comportamento em condições de teste (if):** Pode-se explicar que em C, C++ e JavaScript o código `if (x = y)` compila, enquanto que em Java e C# é exigido que o resultado da expressão seja estritamente um valor booleano.
- **Restrições em Go e Rust:** É possível verificar que em Go nem a atribuição nem operações como `x++` são expressões. Em Rust, a atribuição devolve um tipo vazio `()`, fazendo com que `if x = 5` não compile.
- **Atribuição em Python:** O diapositivo permite demonstrar a atribuição múltipla para trocar valores sem variável temporária (`a, b = b, a`) e explicar que o operador "morsa" (`:=`, a partir da versão 3.8) é a exceção que permite usar uma atribuição como expressão no Python.
- **Padrões na linguagem C:** A imagem fornece exemplos práticos de como a atribuição funciona no meio de cálculos (`a = b + (c = d / b) - 1`), em múltiplos alvos (`sum = count = 0`) ou dentro de um ciclo `while`.
- **Linguagens funcionais:** É abordado o conceito de imutabilidade, indicando que palavras-chave como `val` (em ML) e `let` (em F# e Rust) são usadas para dar um nome a um valor que não pode ser alterado

Slide 14 - Atribuição de modo misto (Sebesta §7.7)

Atribuição de modo misto

Java · só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;         // erro: estreitamento
int i = (int) 3.7;      // 3 (cast explícito)
long l = i;             // ok

i += 3.7;               // COMPILA: cast escondido
System.out.println(i);  // 6
```

C · coerção em qualquer direção

```
int i = 3.7;            // 3, em silêncio
char c = 300;           // 44 (só um aviso)
unsigned int u = -1;     // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para double.

Sebesta, 11ª ed. · §7.8 · p. 321-322 · exemplos/06_atribuicao

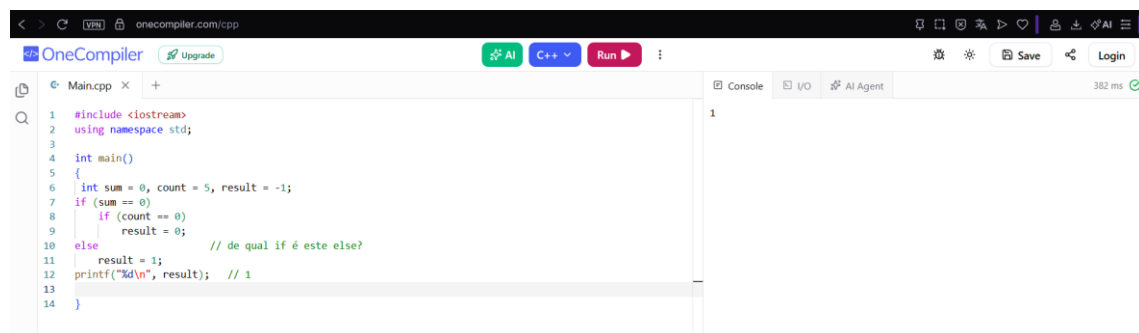
É quando o lado direito da atribuição tem um tipo diferente da variável do lado esquerdo. Por exemplo, `int x = 3.7;` coloca um double dentro de um int. A pergunta é se a linguagem permite isso e converte sozinha.

- **C, C++, Fortran e Perl** permitem e fazem coerção livremente, inclusive com estreitamento. Assim, `int x = 3.7;` compila e x fica com **3**, perdendo a parte fracionária sem aviso.
- **Java e C#** só aceitam coerção de alargamento. `double d = 5;` compila, mas `int x = 3.7;` dá erro, e é preciso escrever `int x = (int) 3.7;`.

- **Ada** não permite atribuição de modo misto: toda conversão tem de ser explícita.
- **Python e outras linguagens dinâmicas** não têm esse problema, porque a variável assume o tipo do valor atribuído.

Em resumo, é o mesmo dilema da coerção em expressões. Permitir o estreitamento é cômodo, mas esconde perda de dados. Aceitar só o alargamento, como faz o Java, é o meio-termo seguro.

Slide 16 - Seleção de dois caminhos: o "else pendente"



O problema: em ifs aninhados com um único else, não fica claro a qual if ele pertence.

java

```

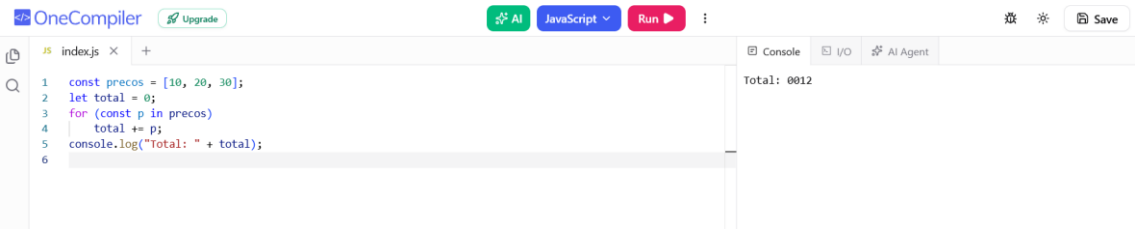
if (soma == 0)
    if (cont == 0)
        resultado = 0;
else // é de qual if?
    resultado = 1;

```

As soluções:

- **C e Java:** o else fica com o if mais próximo, aqui o if (cont == 0), apesar da indentação. Para mudar isso, use chaves.
- **Perl:** chaves são obrigatórias, então não há ambiguidade.
- **Ruby e Ada:** cada if termina com end, o que deixa os limites explícitos.
- **Python:** a indentação define o bloco.

Slide 22 - Exercício em duplas: encontre o bug (1/3)



20 - Desvio incondicional: o got

Desvio incondicional: o goto

"goto fail" (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail; // duplicada: desvia SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail; // nunca é alcançada
fail:
    return err; // 0 = "certificado válido"
}
```

Dijkstra (1968)
"Go To Statement Considered Harmful"

Quem tem goto hoje
C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos
break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. · §8.4 · p. 352-353 · exemplos/09_desvio_protegido

Foi analisado o comando goto e o caso real do erro conhecido como “goto fail”.

Slide 24 - encontre o bug (3/3)

onecompiler.com/go

OneCompiler

Upgrade

AI

Go

Run

Save

Login

272 ms

main.go

```
1 package main
2 import "fmt"
3
4 func main() {
5     x := 10
6     if x > 5 {
7         x := x * 2
8         fmt.Println("dentro:", x)
9     }
10    fmt.Println("fora:", x)
11
12 }
```

Console

I/O

AI Agent

dentro: 20
fora: 10